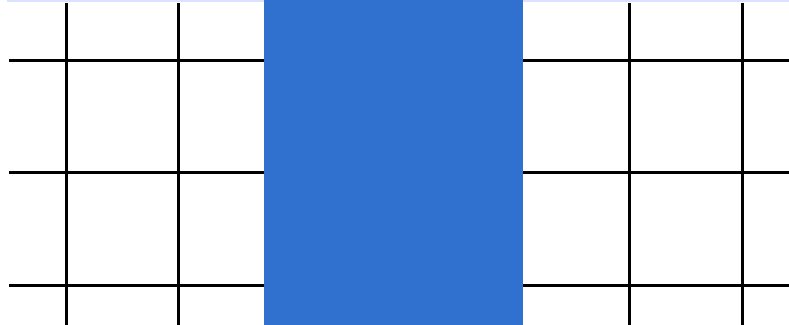
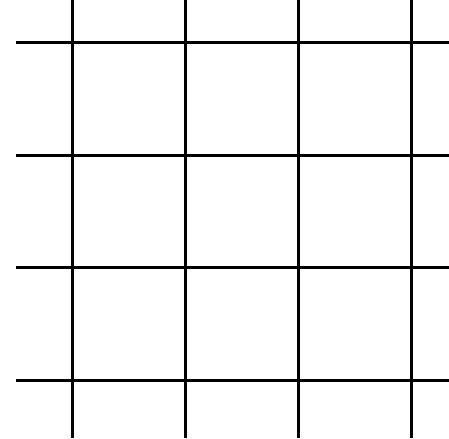
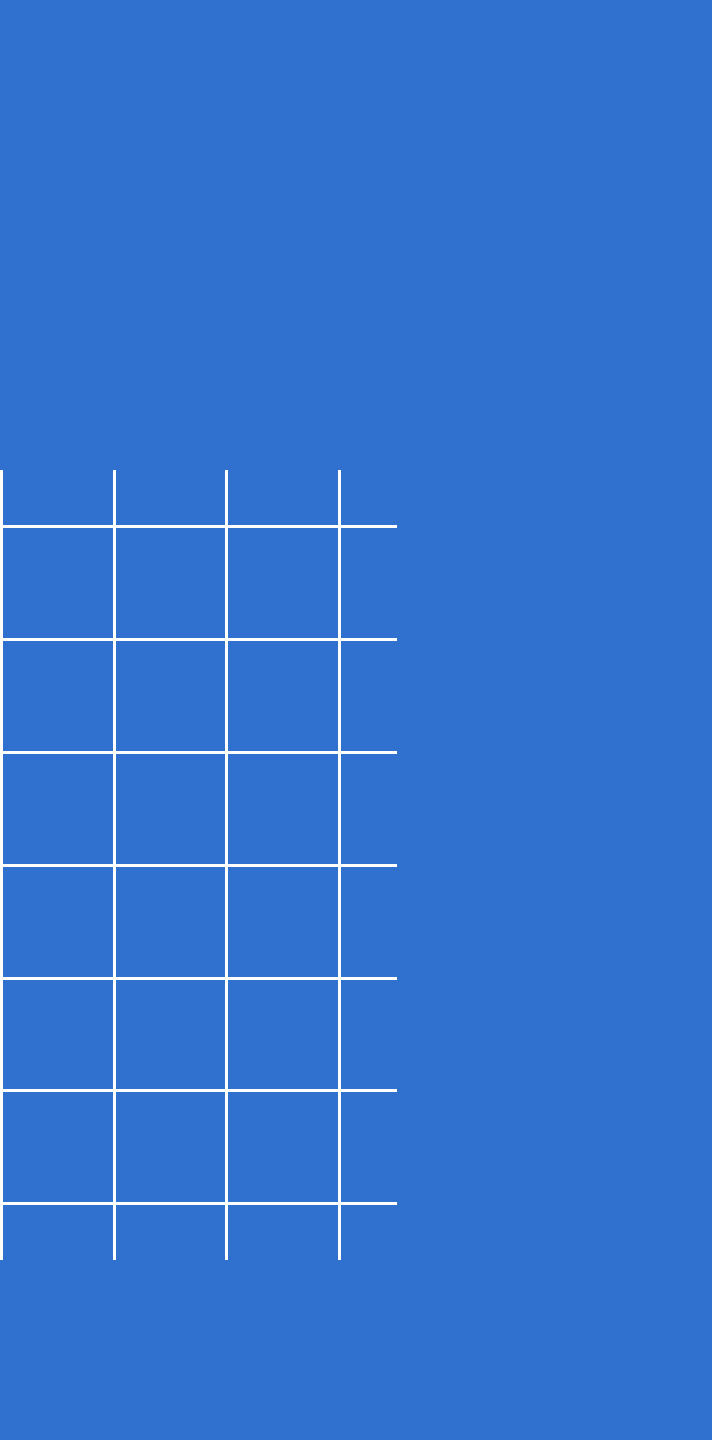


# INTRO GENERATIVE ADVERSARIAL NETWORK (GAN)

– Matee Vadrukchid –





# Part 1: Autoencoders

## Part 1: Autoencoders

- Auto = self
- Encode = convert into a different form

Autoencoder = a system that teaches itself how to encode information

It is a model that teaches itself how to encode information

## Part 1: Autoencoders

- An unsupervised learning technique that is used as a data representation
- The idea is to use CNN to act as data compression/data encoding by introducing a bottleneck layer
- We must have encoding layers and decoding layers

# Part 1: Autoencoders

## Loss function

Binary Cross Entropy/Log  
Loss

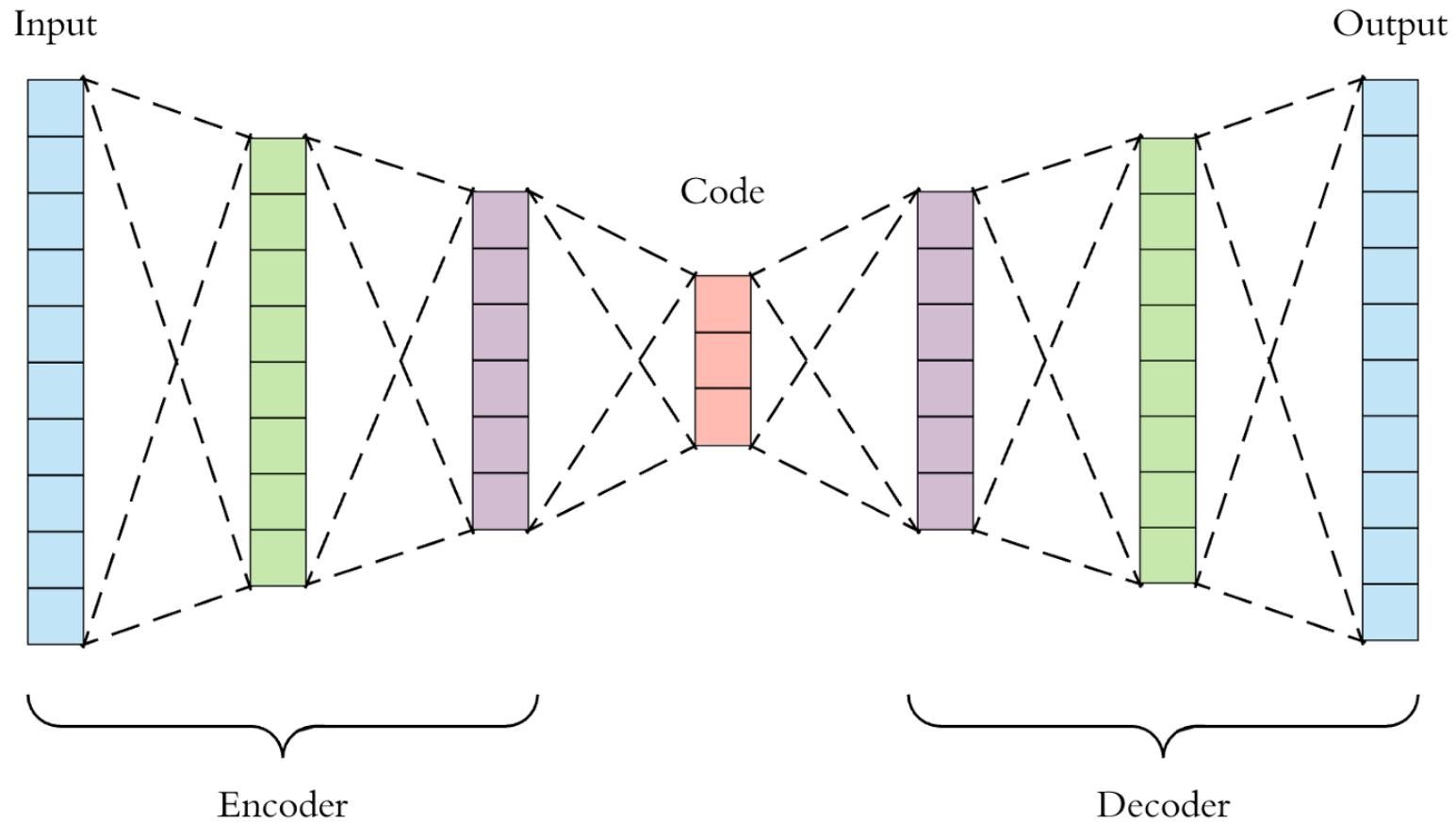
$$BCE = -\frac{1}{n} \sum_{j=1}^n \sum_{i=1}^c [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

MSE  
Loss

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

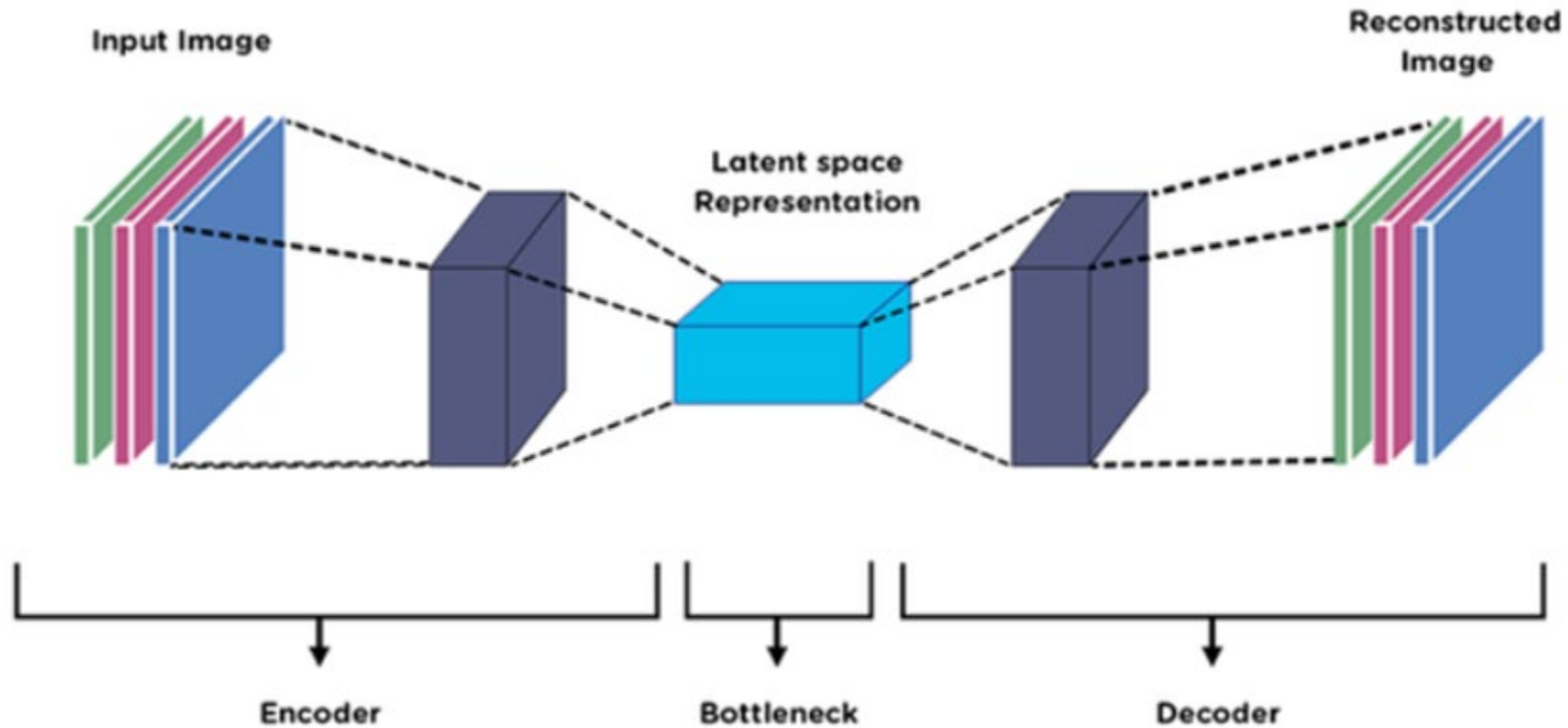
# Part 1: Autoencoders

## Example



## Part 1: Autoencoders

# CNN Autoencoder



## Part 1: Autoencoders

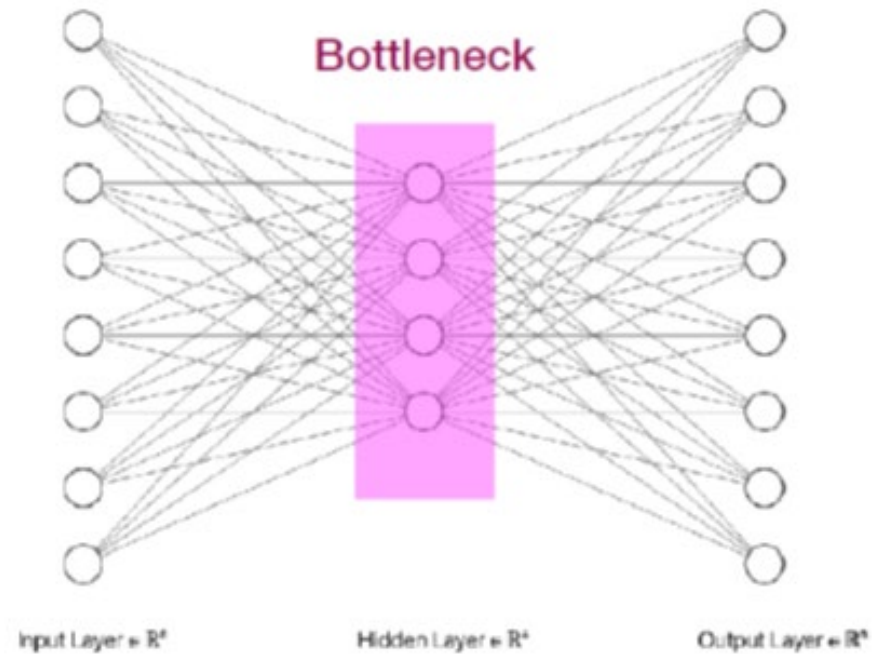
# Autoencoder applications

- Denoising
- Fix Image Inpainting
- Information Retrieval
- Anomaly Detection



## Part 1: Autoencoders

- Introduce a bottleneck layer to compress the data



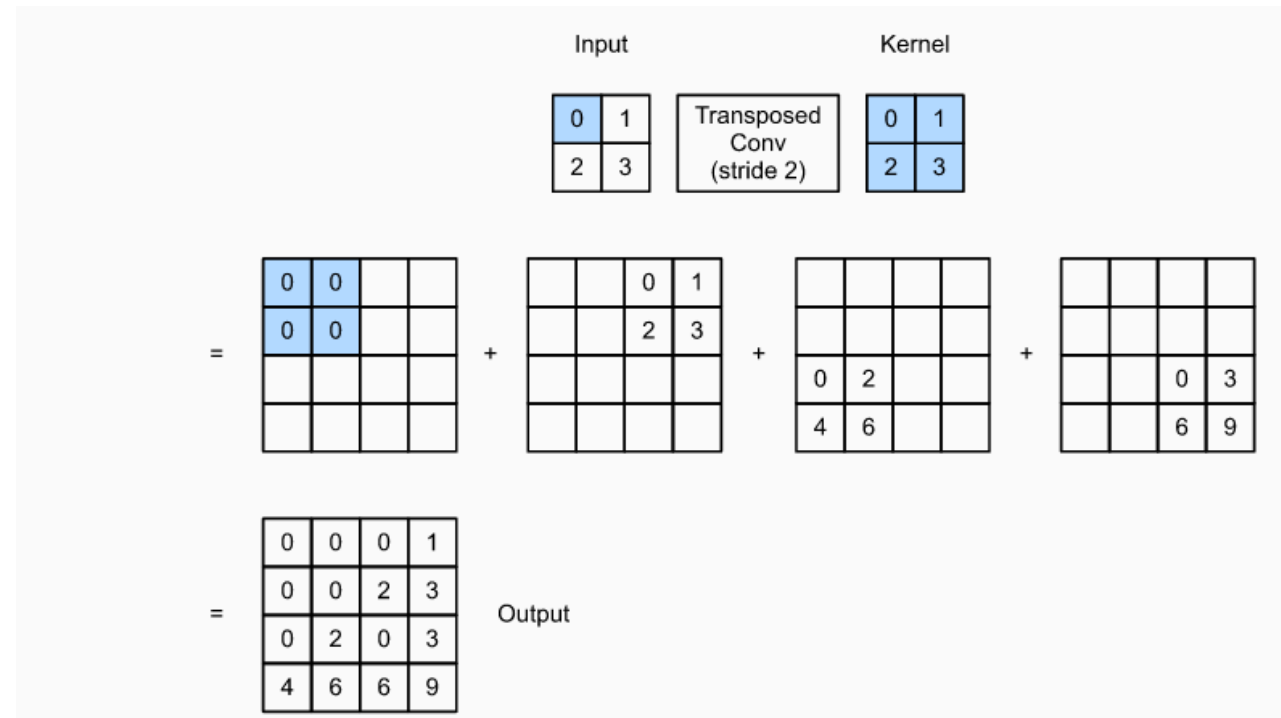
## Part 1: Autoencoders

### Terminology

- Convolution with stride  $\geq 2$  (downsampling)
- Transpose convolution (upsampling)

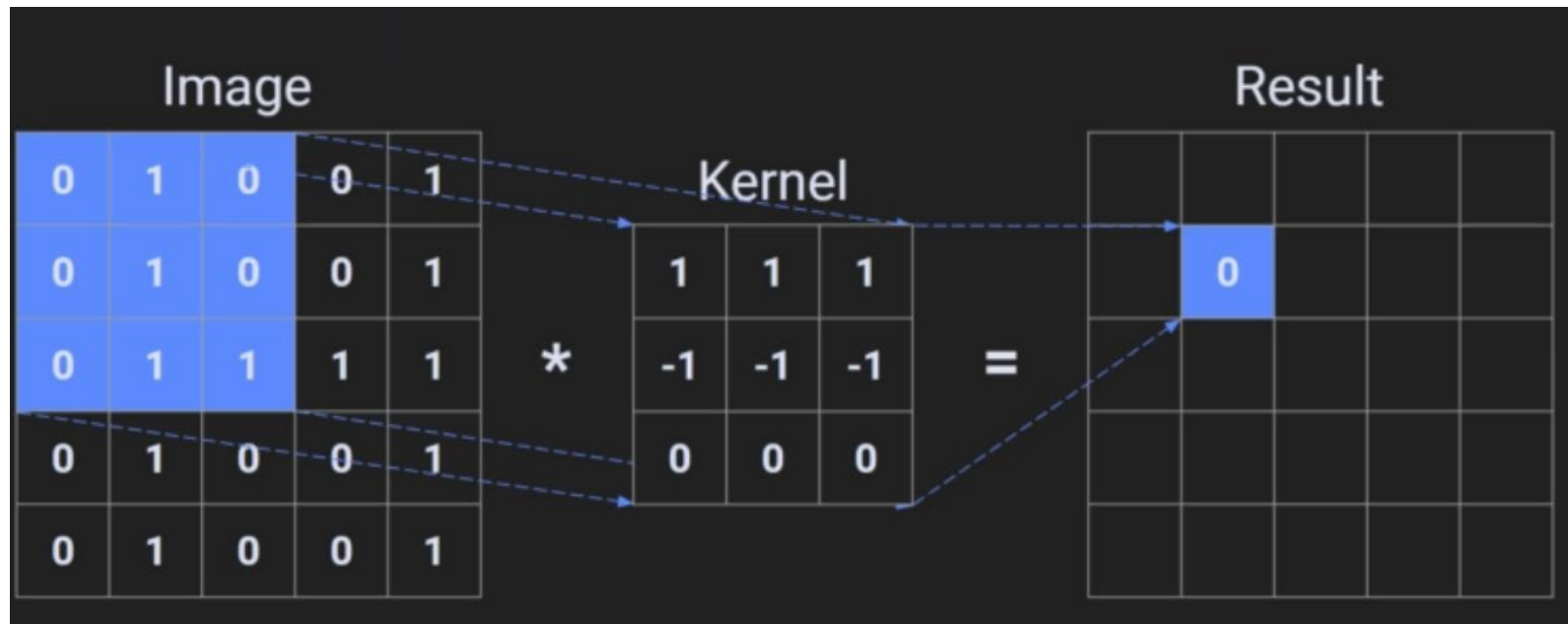
## Part 1: Autoencoders

# Convolution/Transposed Convolution



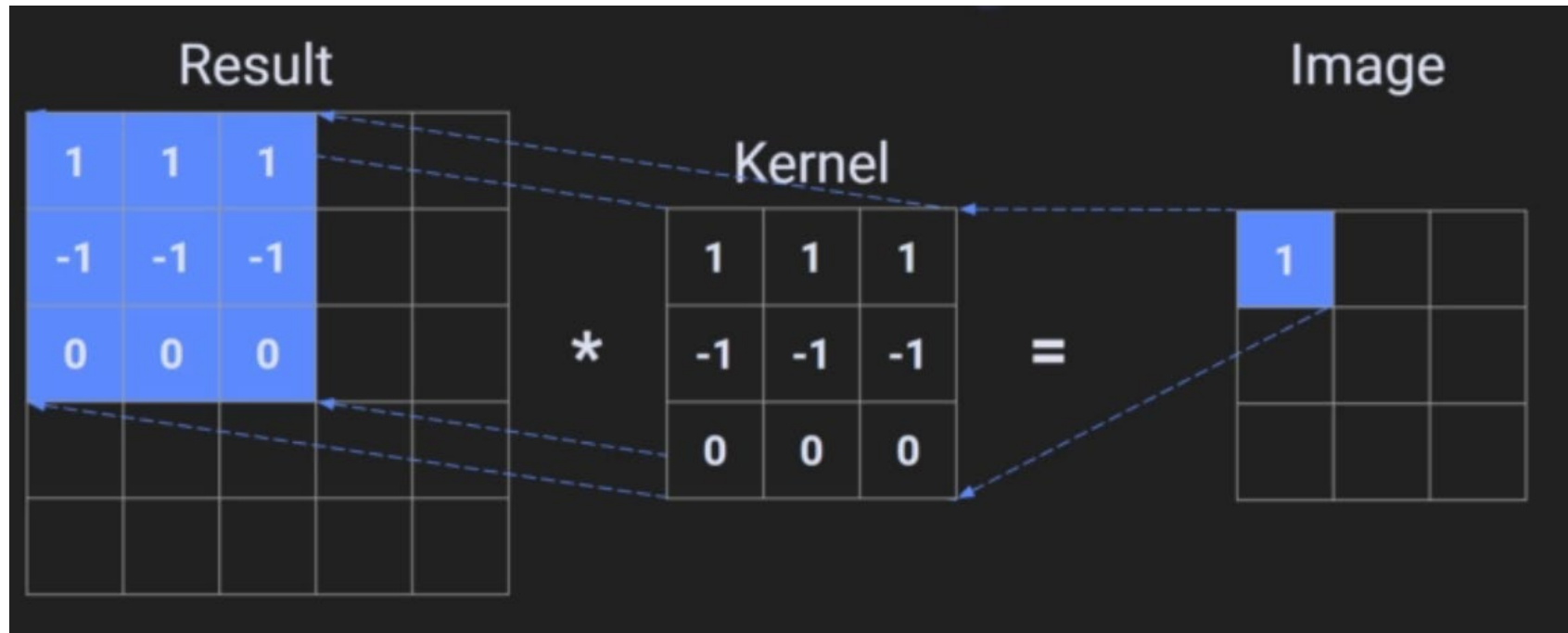
## Part 1: Autoencoders

### Convolution



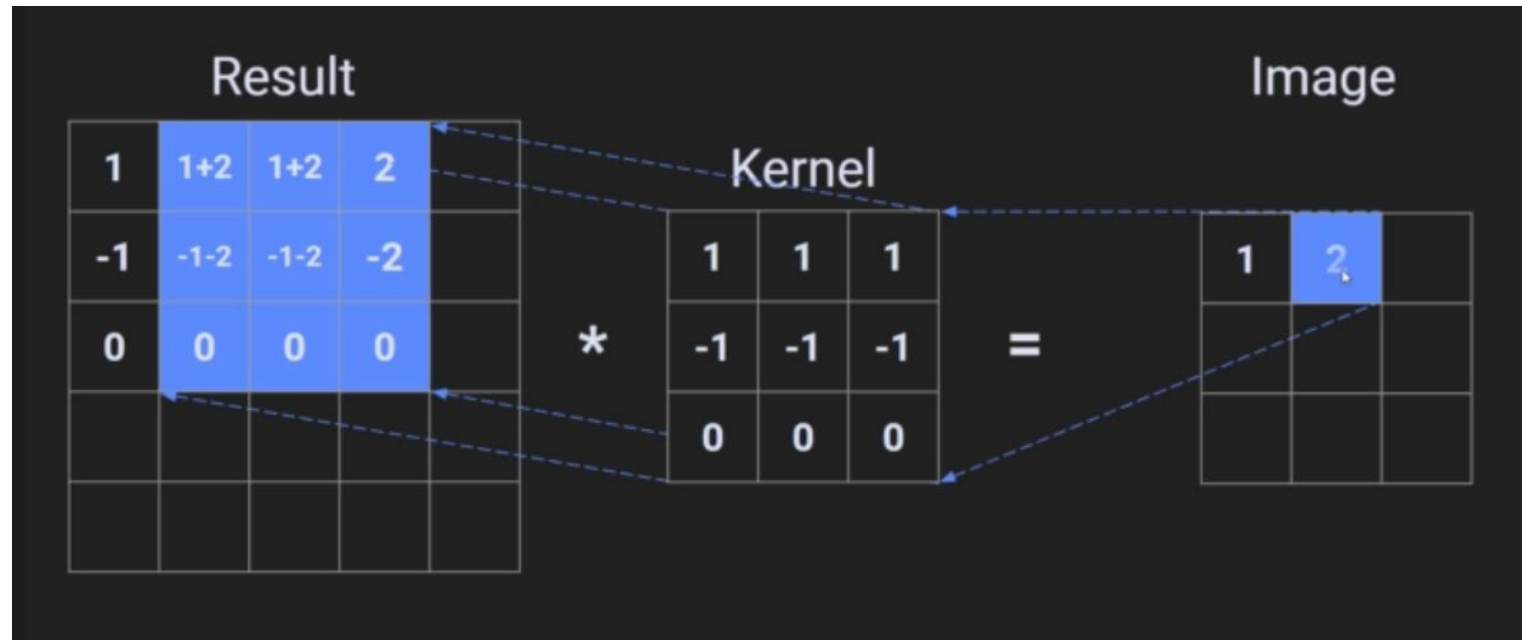
# Part 1: Autoencoders

## Transpose Convolution

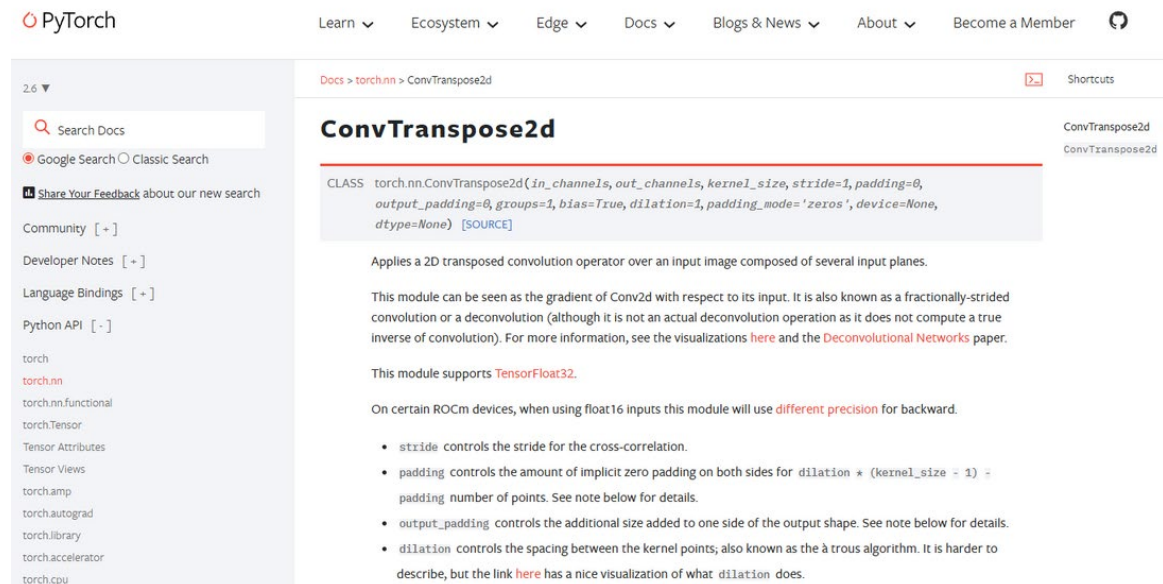


## Part 1: Autoencoders

# Next Step



# Part 1: Autoencoders



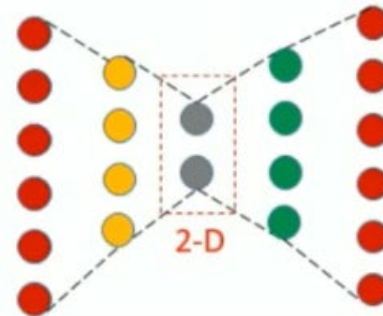
$$H_{out} = (H_{in} - 1) \times \text{stride}[0] - 2 \times \text{padding}[0] + \text{dilation}[0] \times (\text{kernel\_size}[0] - 1) + \text{output\_padding}[0] + 1$$

$$W_{out} = (W_{in} - 1) \times \text{stride}[1] - 2 \times \text{padding}[1] + \text{dilation}[1] \times (\text{kernel\_size}[1] - 1) + \text{output\_padding}[1] + 1$$

# Part 1: Autoencoders

## The number of latent variables

- The number of latent variables (the number of output neurons/dimension) of the last encoder layer/the compressing layer (the dimensionality of the compressed representation) matters. The more, the better:



| Input image | 2-D latent space | 5-D latent space | 10-D latent space | 20-D latent space |
|-------------|------------------|------------------|-------------------|-------------------|
|             |                  |                  |                   |                   |



# Part 1: Autoencoders

## Deep Fake with AutoEncoder



*Donald Trump → Mr. Bean*

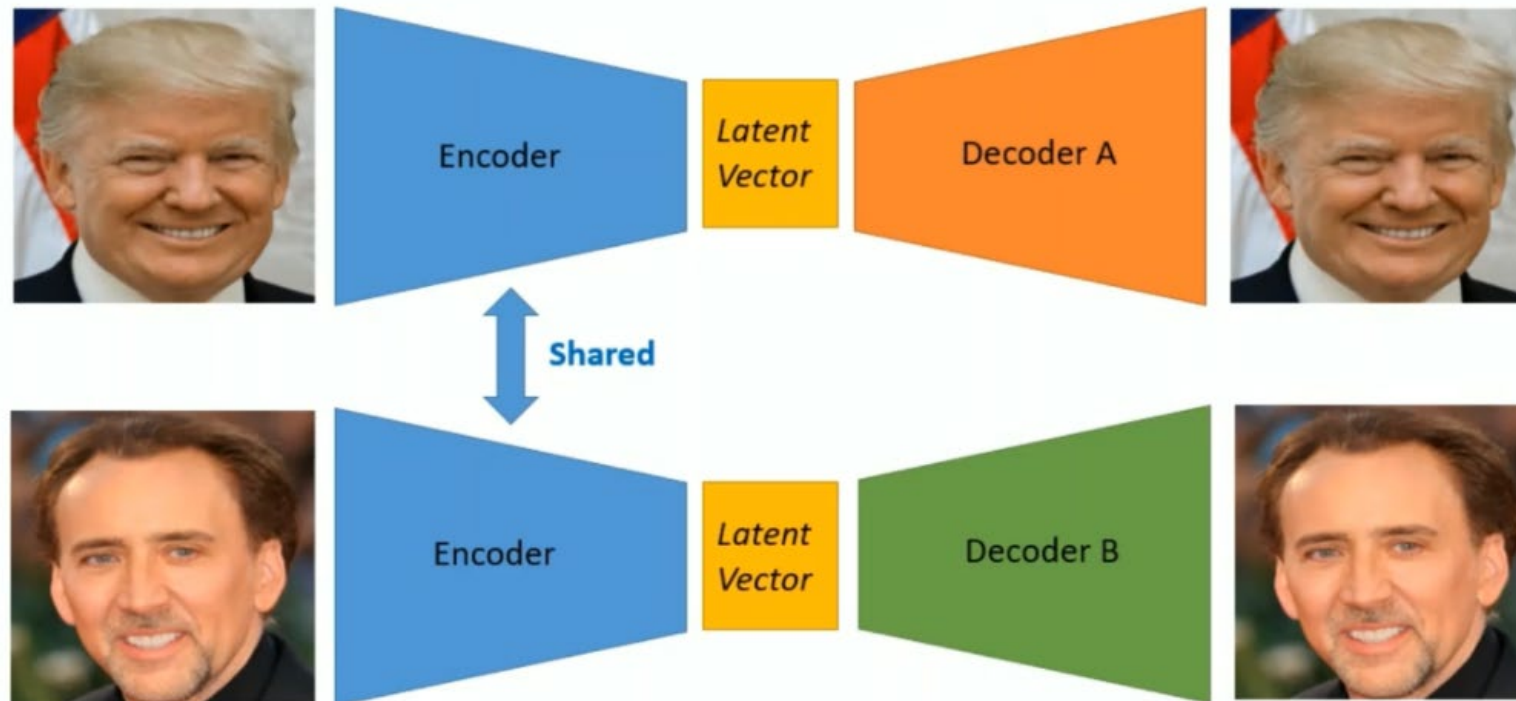


*Home Alone → Home Stallone*

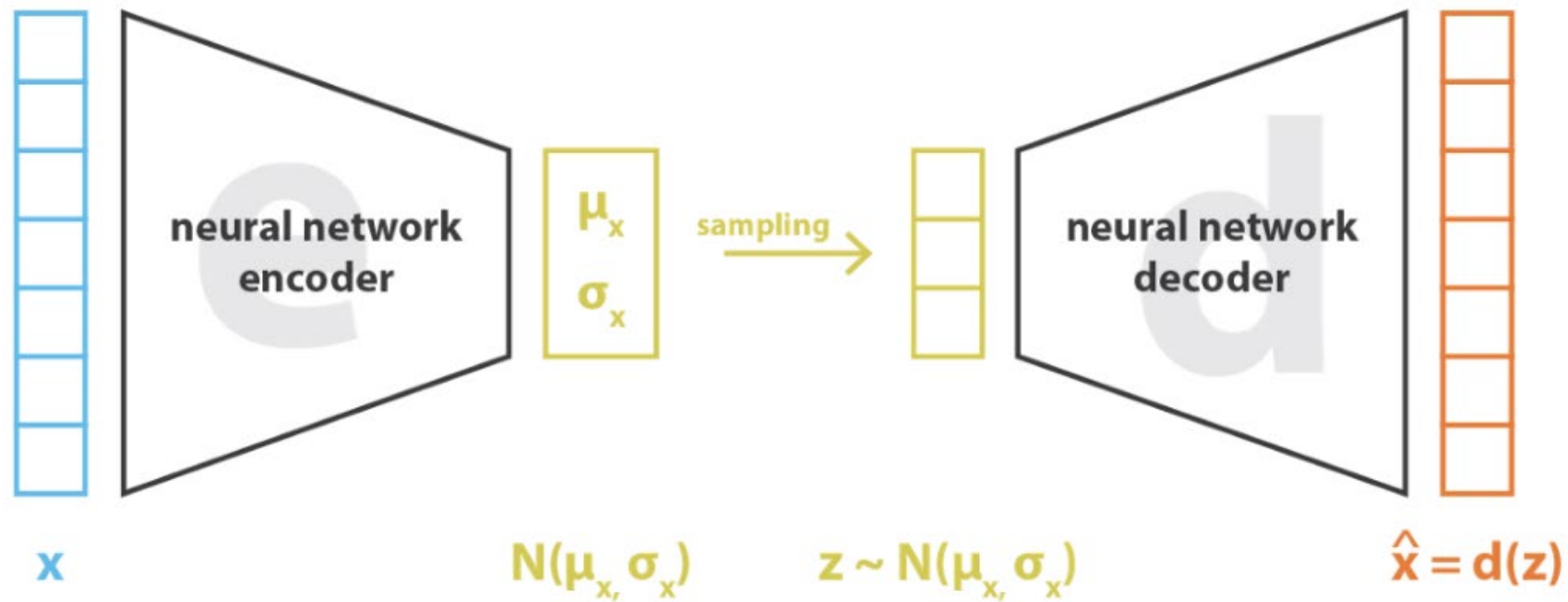
# Part 1: Autoencoders

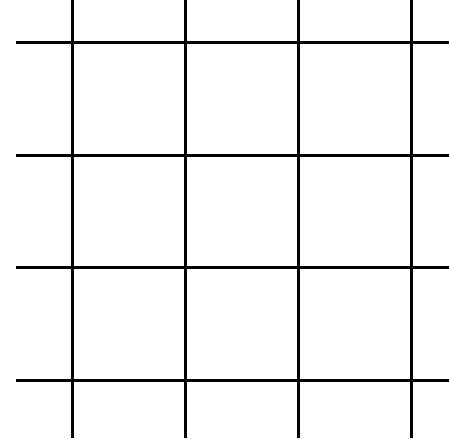
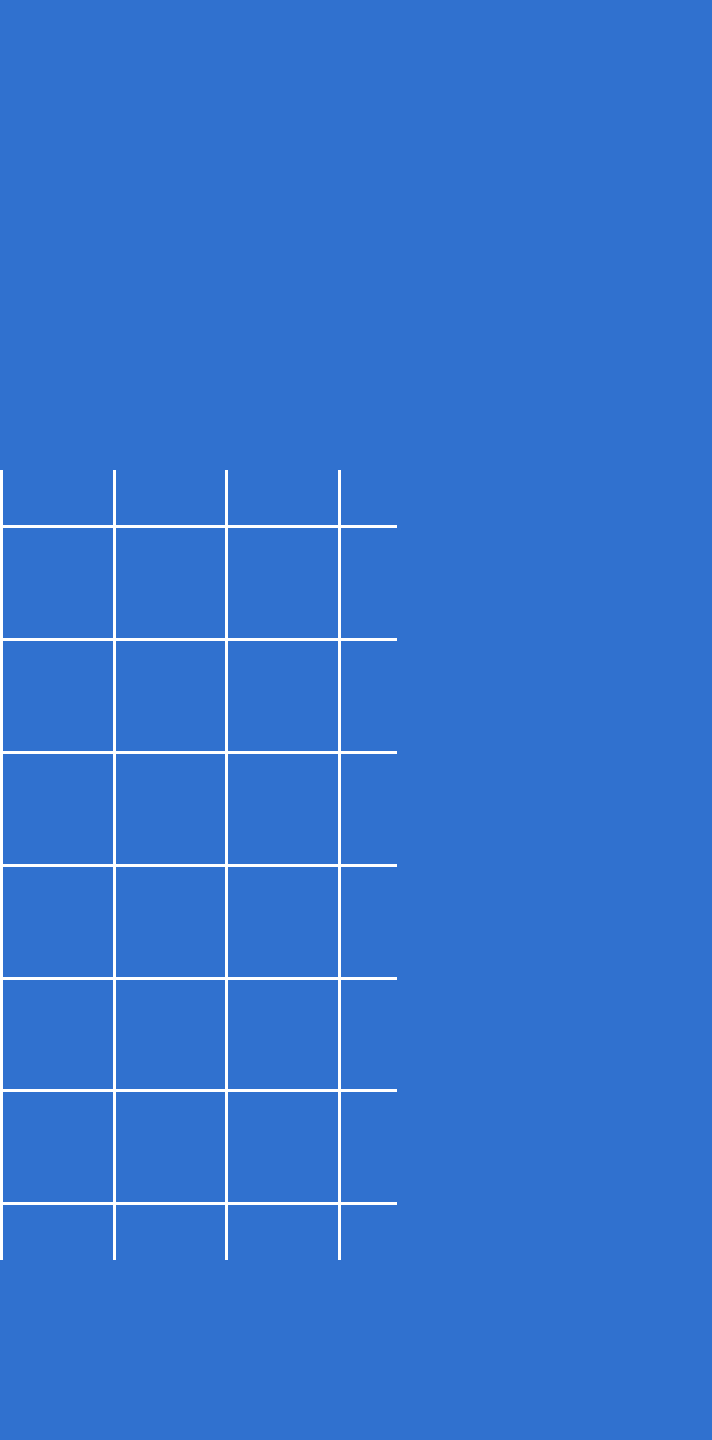
## Training Phase

The **Decoder A** is only trained with faces of A; the **Decoder B** is only trained with faces of B. However, all latent faces are produced by the **same Encoder**.



## Part 1: Autoencoders



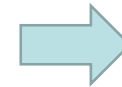
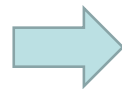


## Part 2: Intro GAN

## Part 2: Intro GAN

# Discriminative models

- CNN classify images



CAT

## Part 2: Intro GAN

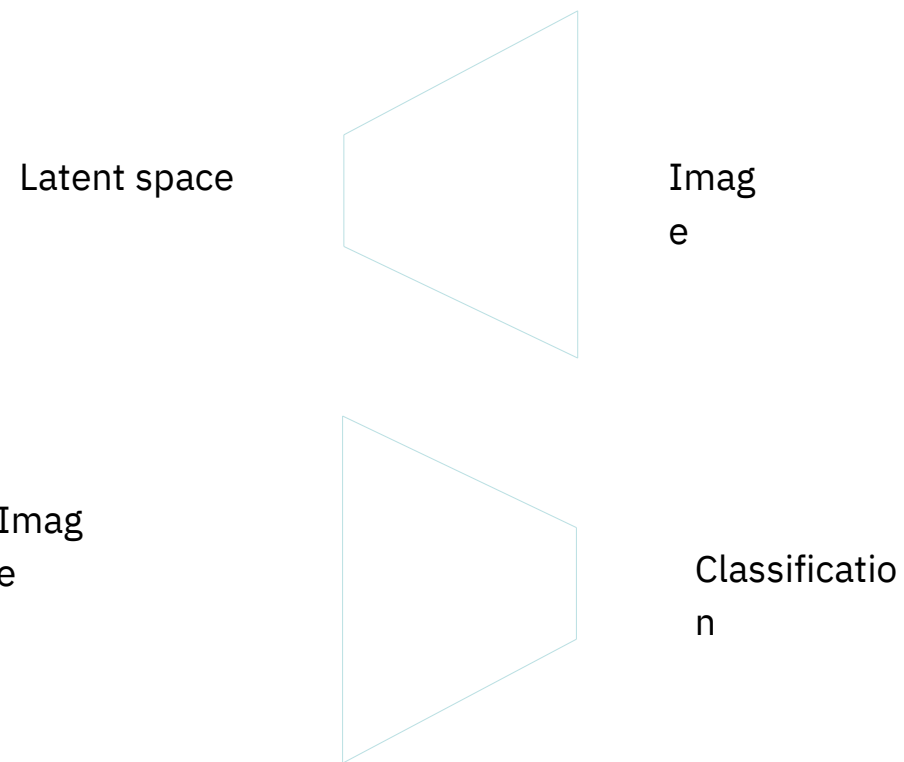
# Generative models

- Create new samples



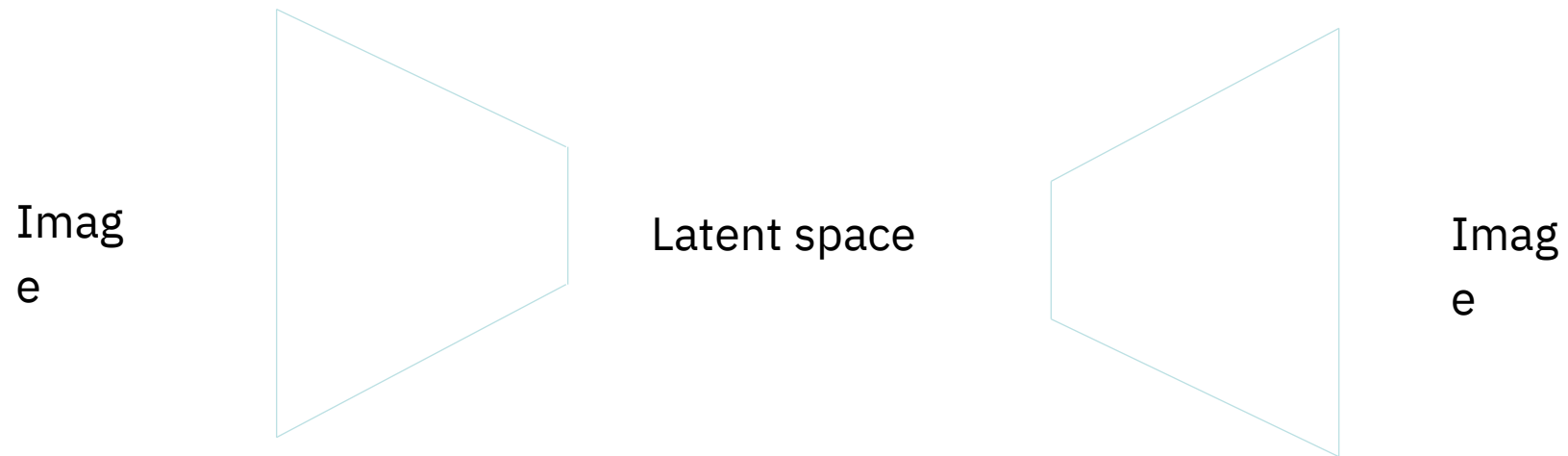
## Part 2: Intro GAN

### More details



## Part 2: Intro GAN

# AutoEncoder





## Part 2: Intro GAN

# GAN

- It is introduced by Ian Goodfellow et al. in 2014
- It is the NN model that generates data from an existing distribution of samples thru loop approach

May 25, 2018, 08:15am EDT

### The Best Tech Innovations Of The Last Three Years



**Forbes Technology Council** COUNCIL POST | Paid Program  
Innovation

POST WRITTEN BY

**Forbes Technology Council**

Successful CIOs, CTOs & executives from **Forbes Technology Council** offer firsthand insights on tech & business.

## Part 2: Intro GAN

# Image-to-Image translation

- Night time from Day time image



## Part 2: Intro GAN

# Super Resolution



Figure 2: From left to right: bicubic interpolation, deep residual network optimized for MSE, deep residual generative adversarial network optimized for a loss more sensitive to human perception, original HR image. Corresponding PSNR and SSIM are shown in brackets. [4× upscaling]

## Part 2: Intro GAN

# ARCANE GAN



## Part 2: Intro GAN

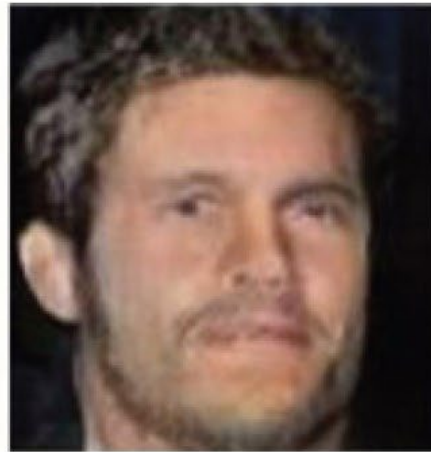
# Generation



2014



2015



2016



2017

# Issues with GANs

- Sensitive to model architecture and features
- Training time
- Evaluating GANs is mostly quantitative
- Difficult to bring research lab to real-world applications
- Mode collapse: Generator only output good knows results (not enough varieties)



## Part 2: Intro GAN

# Example



## Part 2: Intro GAN

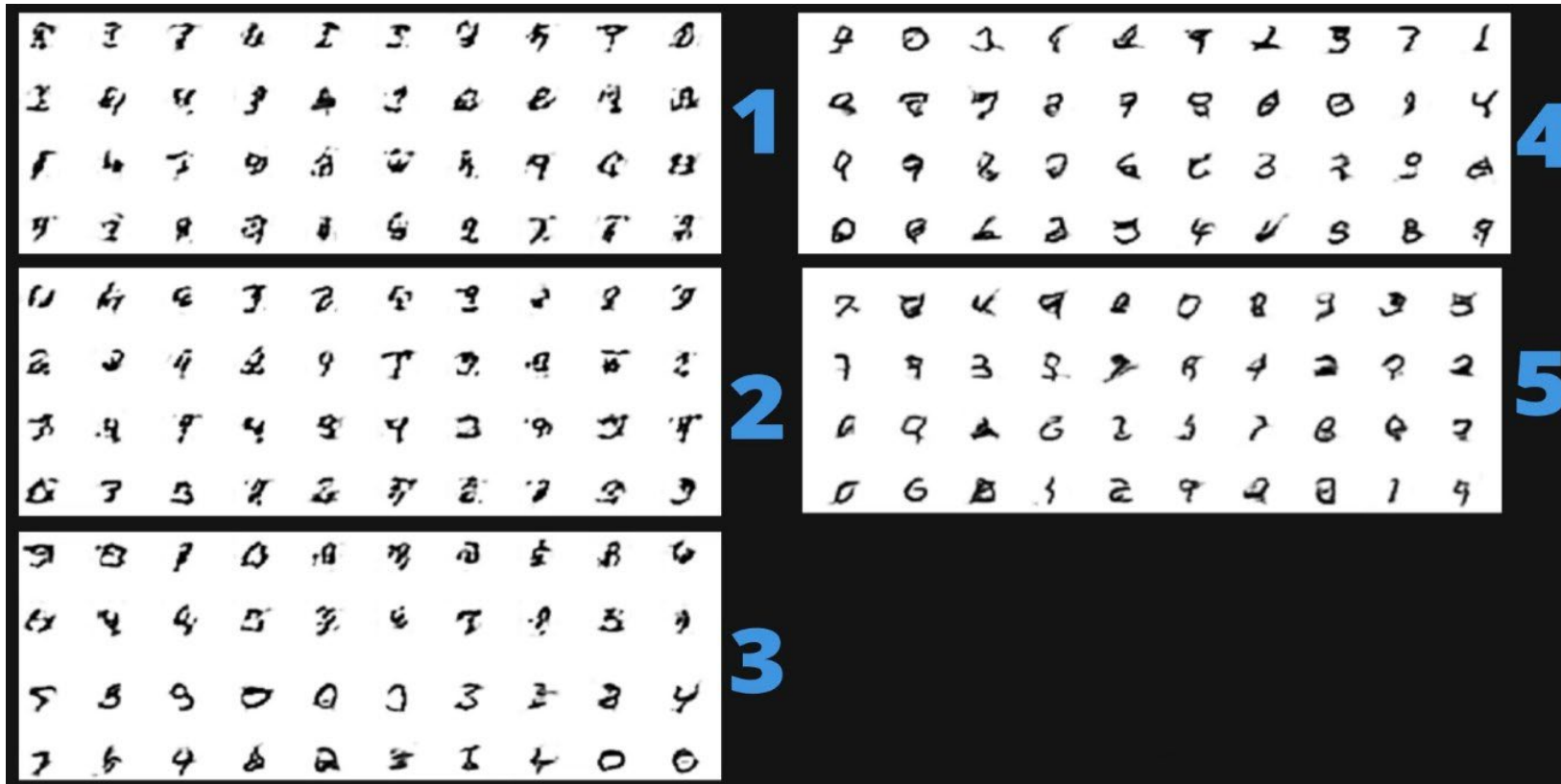
# Training Process

- Randomly generate a noisy vector
- Input this into generator network to create samples
- Take some sample and mix with some real data
- Train discriminator to classify mixed dataset
- Discriminator predict 0 ( real) or 1 (fake)



## Part 2: Intro GAN

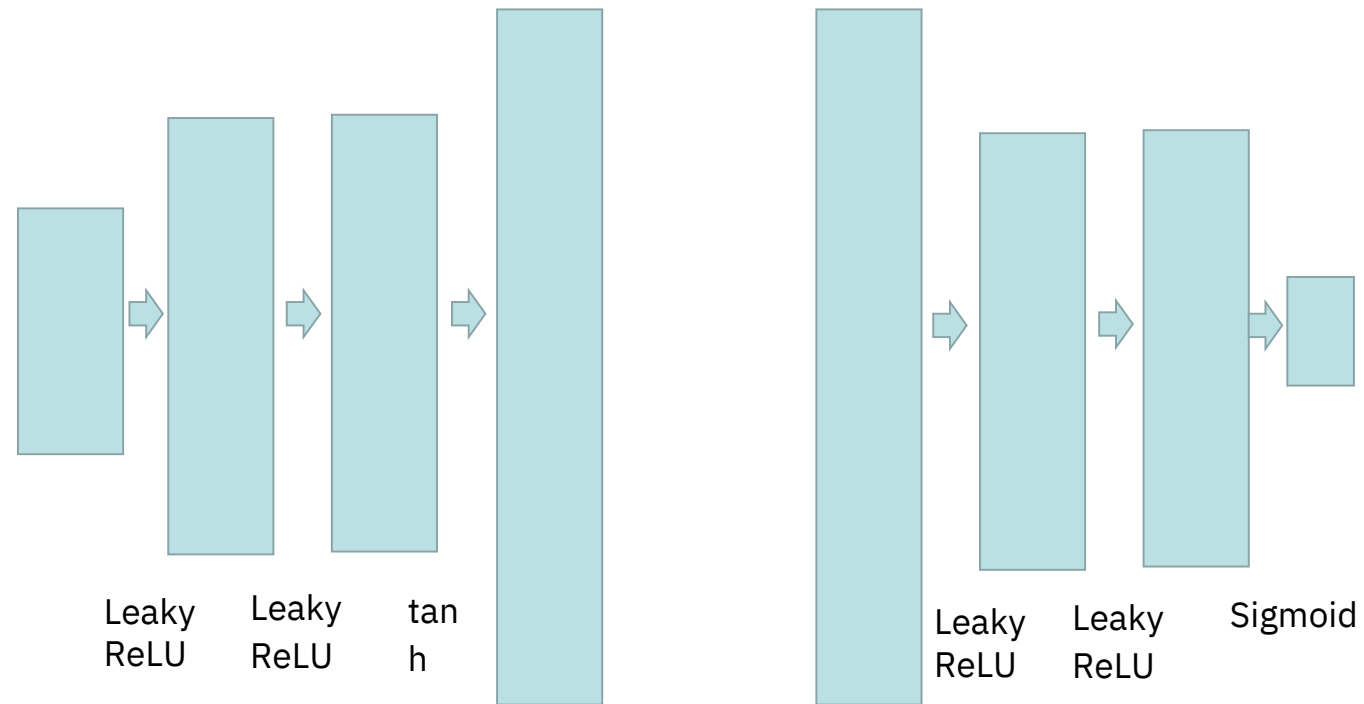
# Generated Sample Data



<https://colab.research.google.com/drive/1N9jCbBtaX-Qf-ZNhTMoVN0ROB6qaZGYI?usp=sharing>

## Part 2: Intro GAN

### FCN GAN

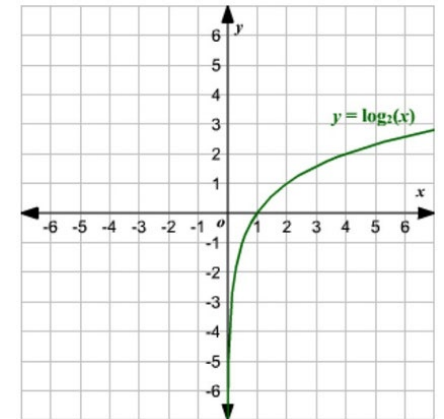


## Part 2: Intro GAN

### Loss Function

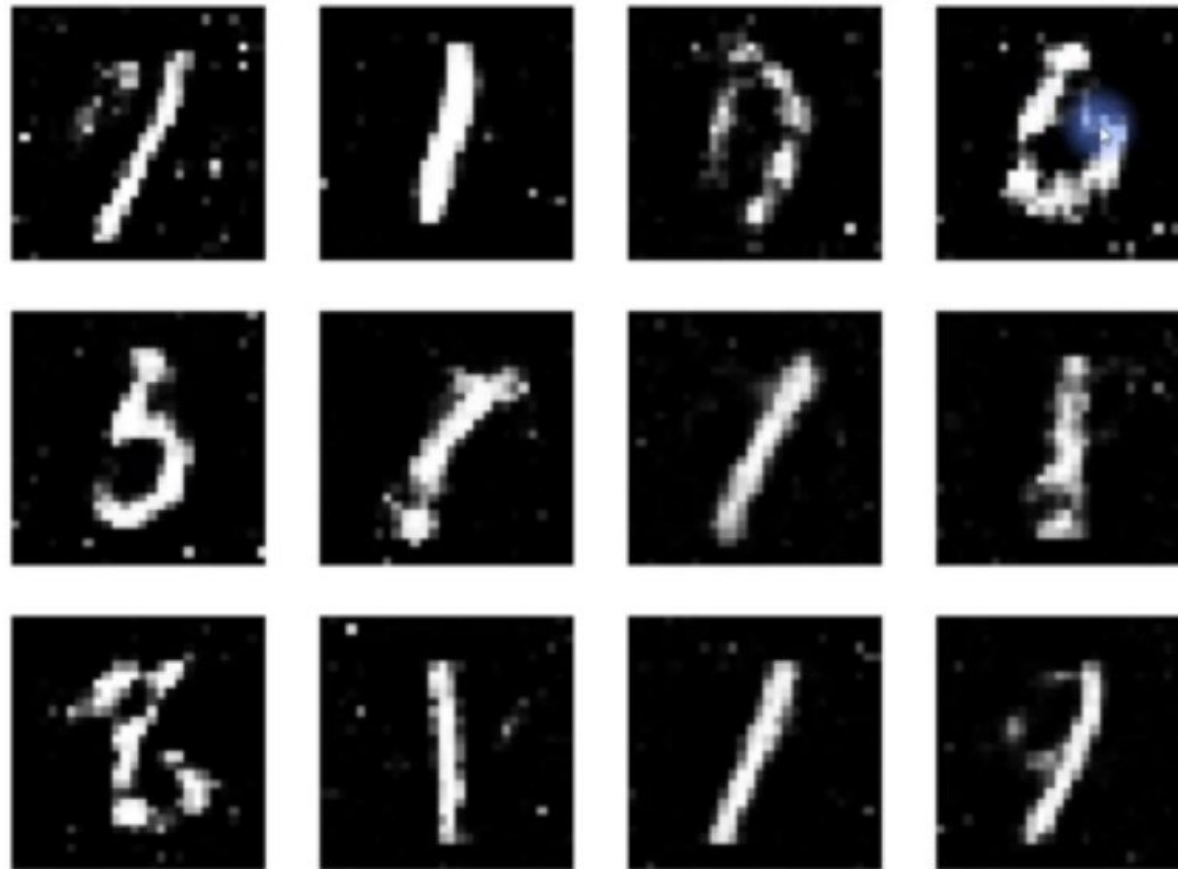
- Discriminator loss:  
we provide ground truth whether the data comes from real or generate images
- Generator loss:  
we use discriminator feedback on fake images to train generator

$$J^D = E_{x \sim p_r} \log[D(x)] + E_{z \sim p_g} \log[1 - D(G(z))]$$



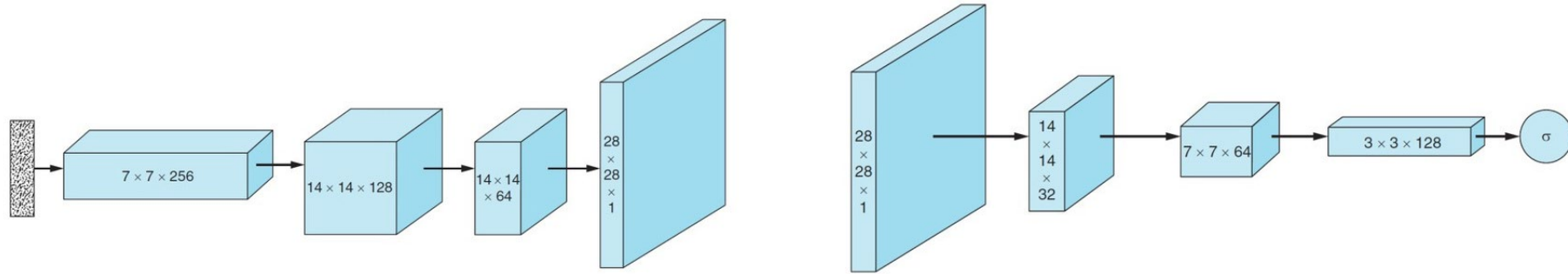
## Part 2: Intro GAN

### Results on MNIST



## Part 2: Intro GAN

# DCGAN



Generator

Discriminator

r

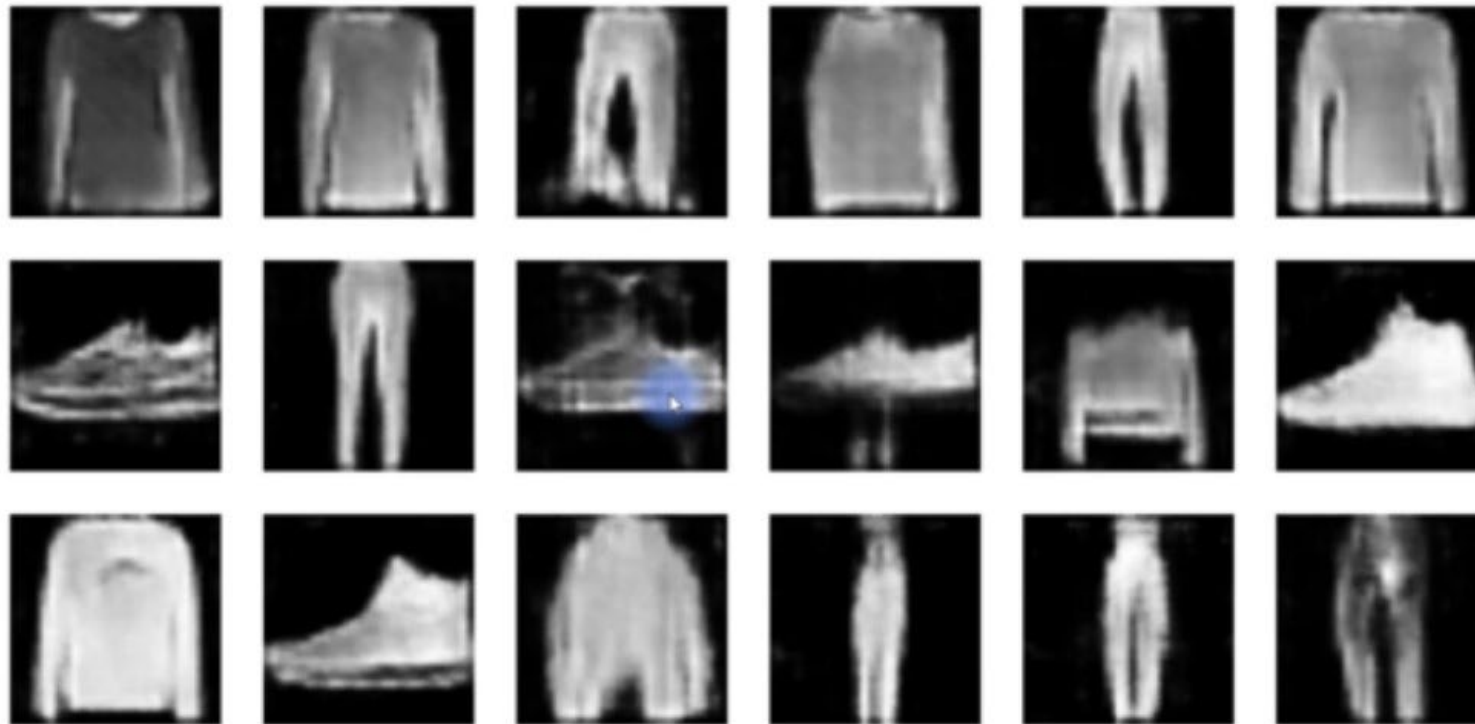
## Part 2: Intro GAN

DCGAN on MNIST



## Part 2: Intro GAN

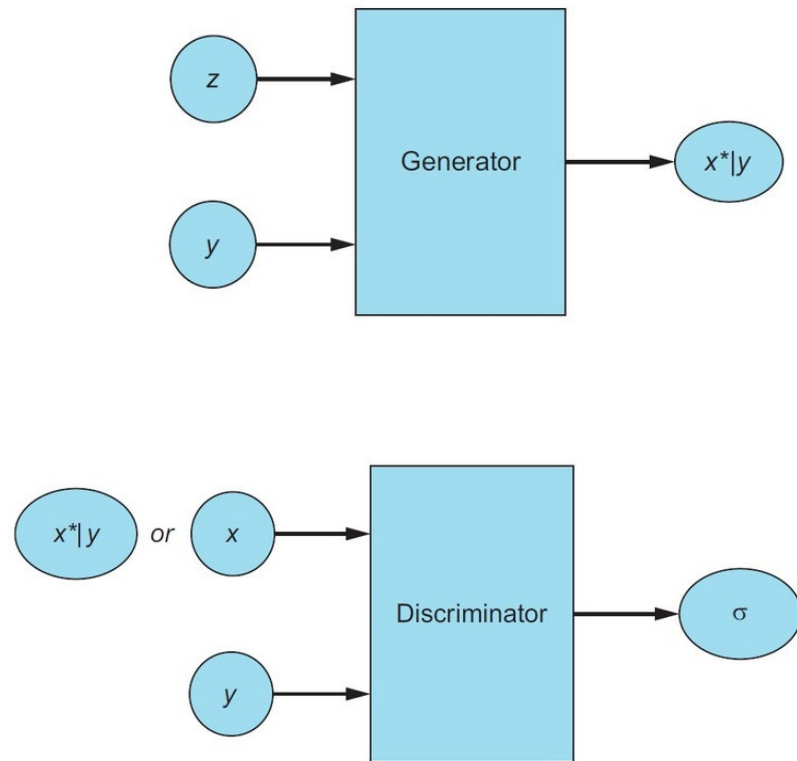
### DCGAN on FMNIST





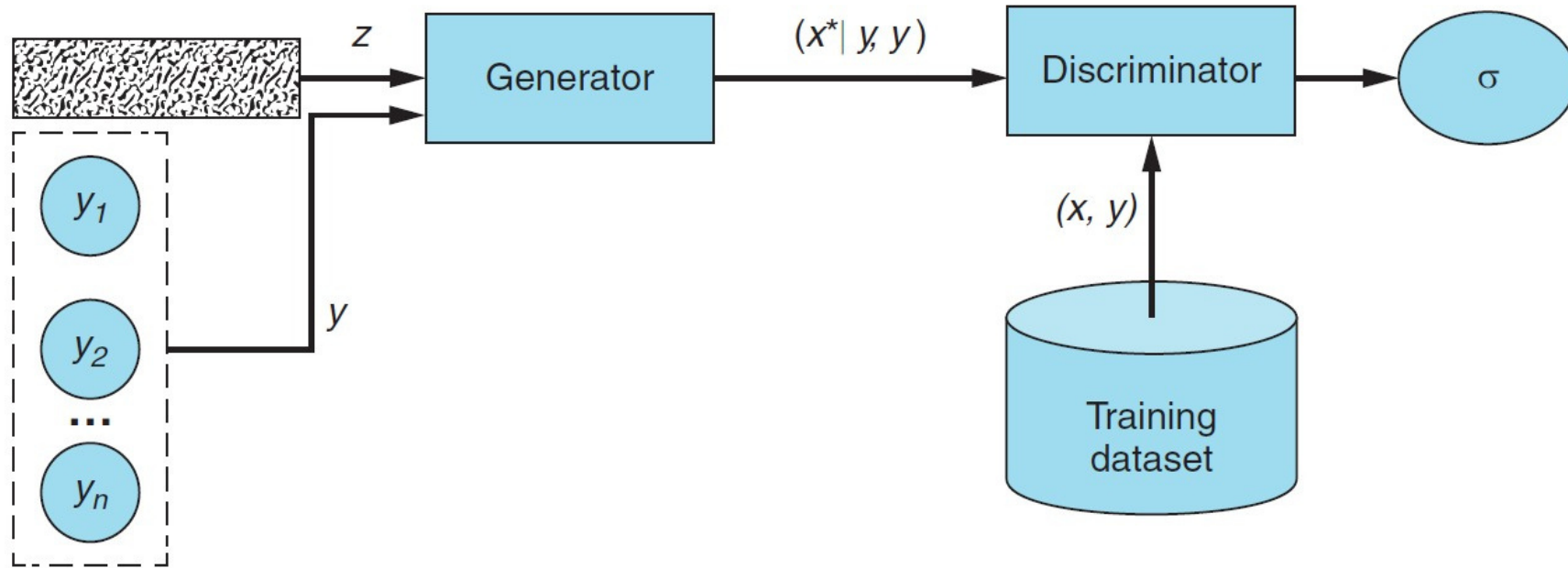
## Part 2: Intro GAN

### Conditional GAN



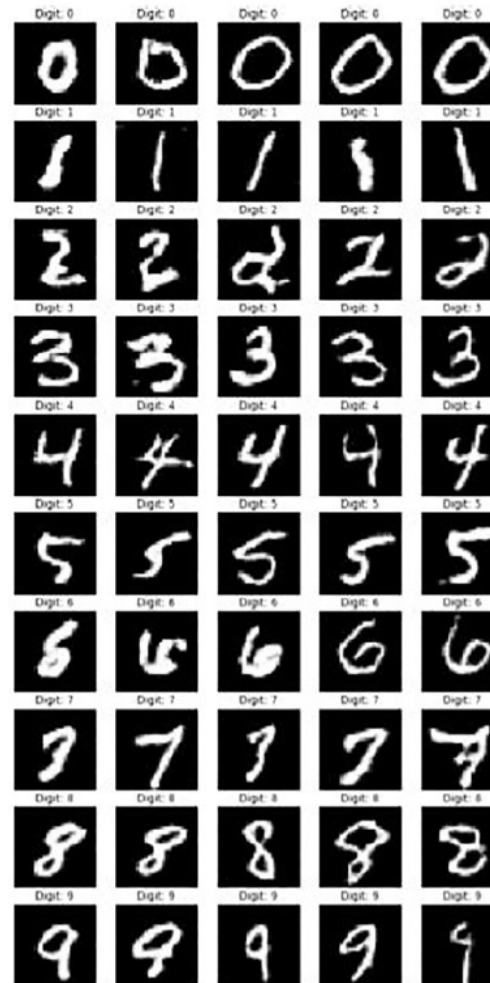
## Part 2: Intro GAN

# CGAN Architecture



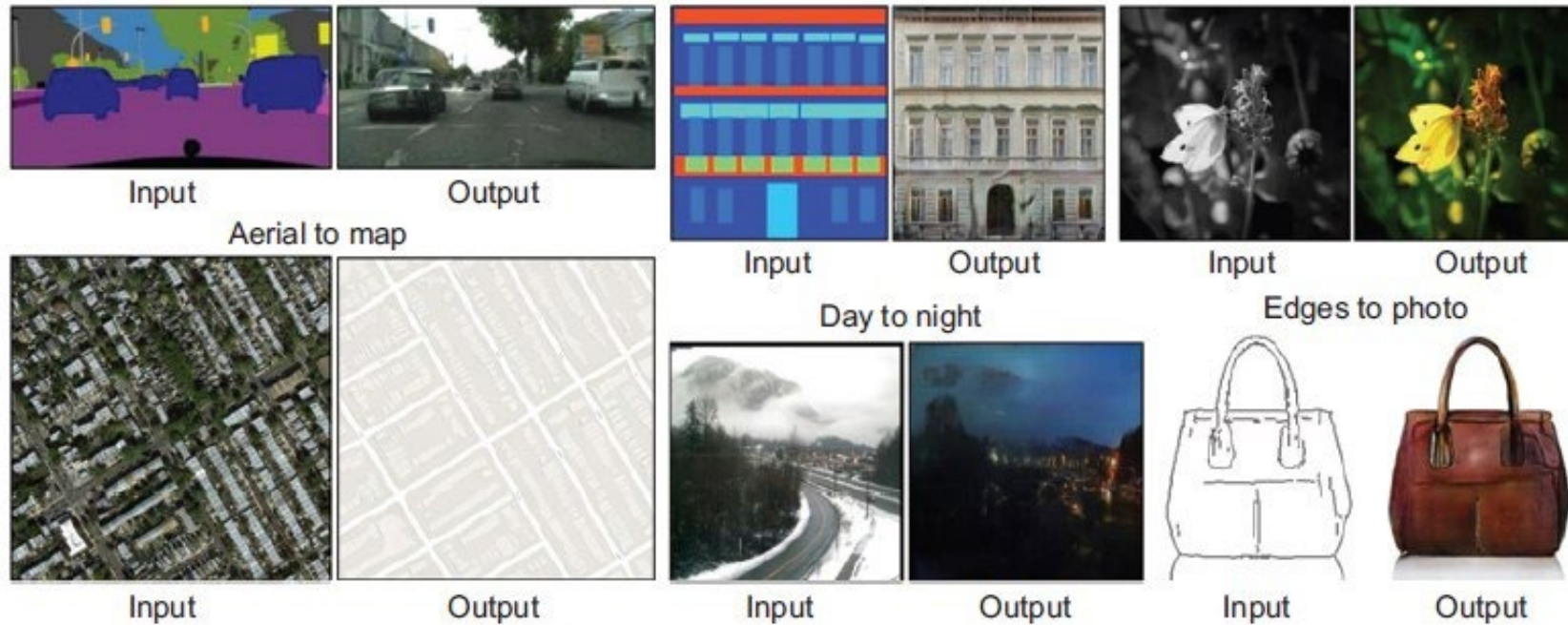
## Part 2: Intro GAN

CGAN  
Output

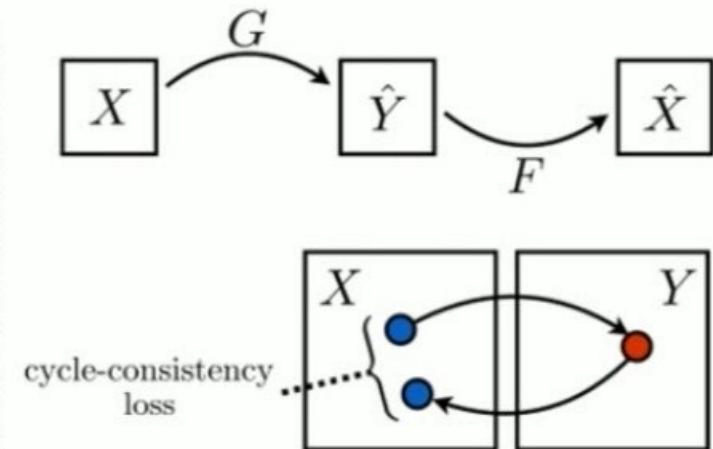
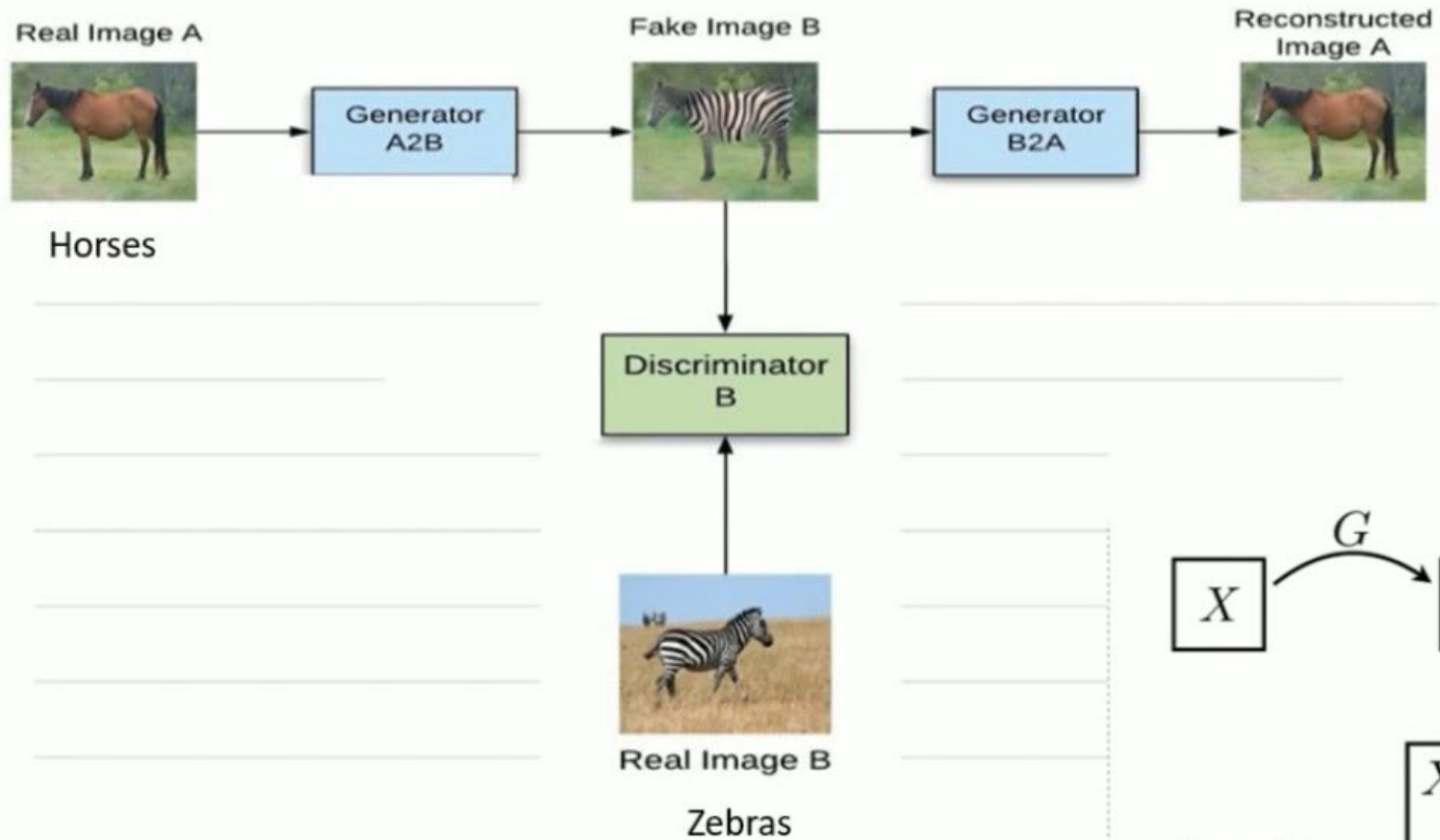


## Part 2: Intro GAN

# CycleGAN Applications

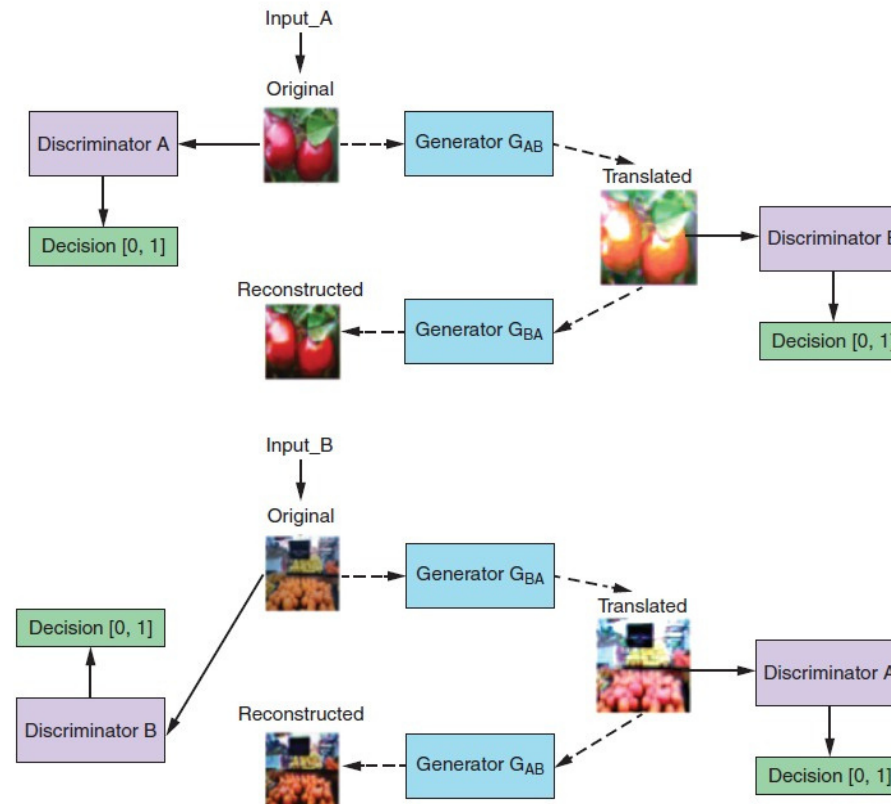


## Part 2: Intro GAN



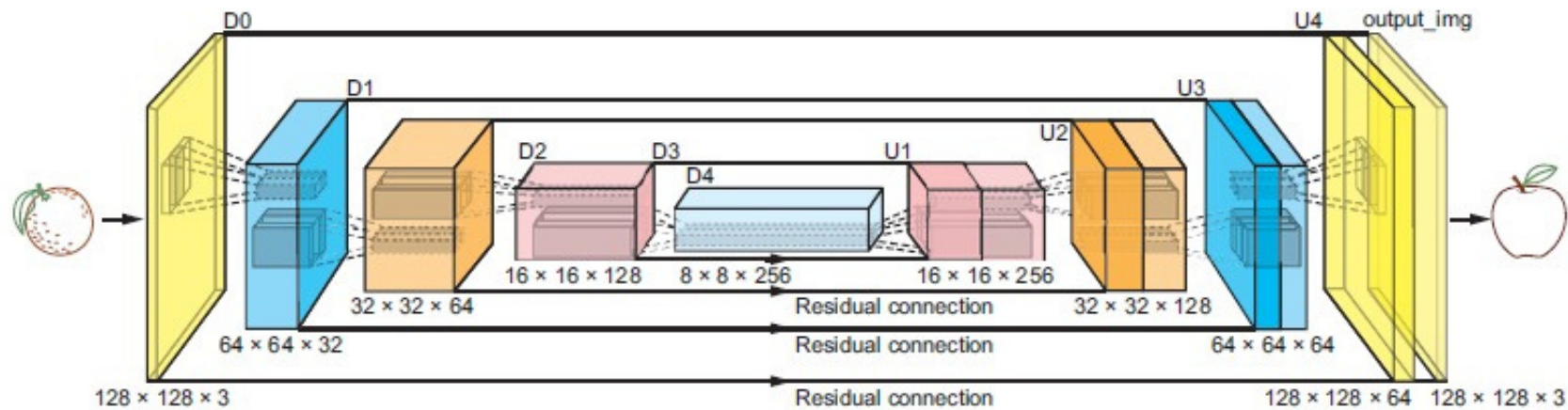
## Part 2: Intro GAN

# CycleGAN



## Part 2: Intro GAN

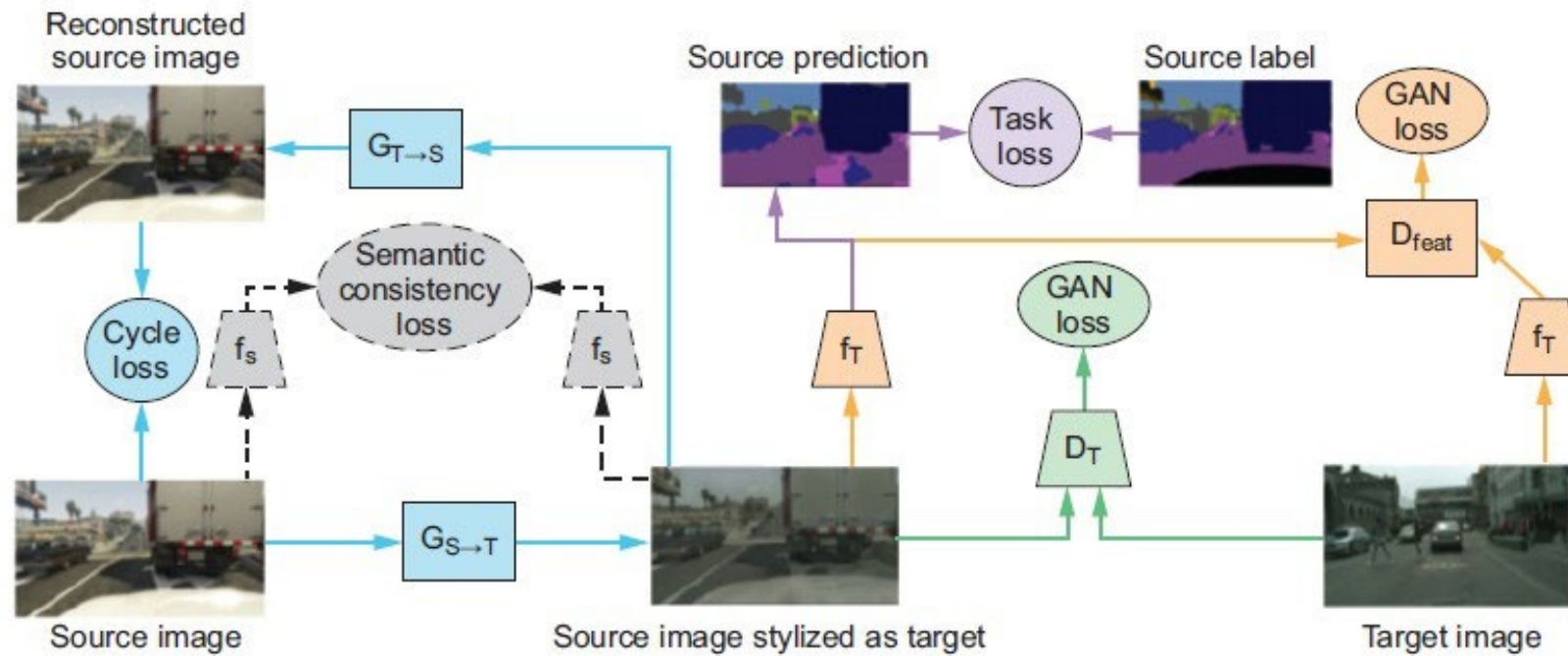
# CycleGAN Architecture





## Part 2: Intro GAN

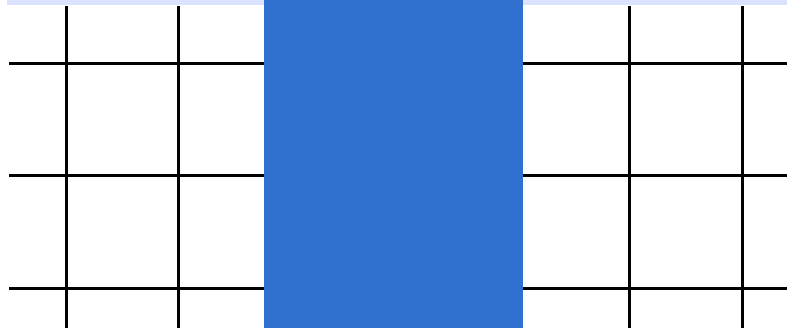
# Output





# INTRO NATURAL LANGUAGE PROCESSING (NLP)

– Matee Vadrukchid –



# Big Picture of NLP

## Big picture of NLP

### Preprocessing

- Tokenization
  - Normal
  - Byte-Pair Encoding
  - WordPiece
  - Unigram
- Numericalization

->

### Embeddings

- Word2Vec
- GloVe
- FastText
- ELMo
- BERT
- SentenceBERT
- SimCSE

->

### Modeling

#### Supervised learning

- Machine learning
- Deep learning
  - RNN
  - LSTM
  - CNN
  - LSTM with attention
- Transformers

#### Self-supervised learning

- BERT - Masked LM
- GPT - LM
- BART, T5, Pegasus - LM

#### Transfer learning

- LLM+ Finetuning / Instruction tuning

#### Prompt-based learning

- LLM + prompt-engineering

#### Augmented-retrieval

- LLM + retrieval

#### Optimization Human

#### Preference learning

- LLM + RLHF

#### Multimodal learning

- LLM + Visual encoder

->

### Inference

#### Probability-based

- Perplexity

#### Word overlap-based

- BLEU
- ROUGE

#### Model-based

- BERTScore
- BLEURT

#### LLM-based

- AlpacaEval
- AlpacaFarm

#### Human Evaluation

#### TASKS

- *Industry* - **Sequence Tagging** (Name Entity Recognition, Part of Speech), **Sentence classification** (sentiment analysis, topic classification), **Image to text (OCR)**, Closed-ended Question Answering, Code Generation, Machine translation **ChatBot** (Retrieval)
- *Research* - Summarization, Intent Detection, Open-ended Question Answering, Coreference Resolution, Music/Story/Poetry Generation (e.g., poetry), Natural language Inference, ASR, VQA

#### TOOLS

- **SpaCy** - basic stuffs - NER, dependency parsing, pos tagging
- **PyTorch** - designing models
- **Huggingface** - pretrained LLM and public datasets
- **Langchain** - toolkit to build context-aware, reasoning LLM applications
- **Deployment** - FastAPI

#### TOP VENUES

1. ACL, NIPS, ICML, ICLR
2. EMNLP, NAACL, EACL
3. TACL, Computational Linguistics, Computer Speech and Language

# 1. Introduction and Outline

## 1.1. What We're Building

- A **mini conversation generator** that responds to simple inputs.
- Example conversation lines:
  1. **User:** "Hello, how are you?"  
**Bot:** "I'm thank you and you?"
  2. **User:** "I'm good." **Bot:** "Great!"
- We'll store a small set of **20 or so** conversation lines (like QA pairs).
- We'll build a neural network model that, given an **input sentence**, generates the **output sentence** token by token, starting with a **special start token** and ending with a **special end token**.

# 1. Introduction and Outline

## 1.2. Lesson Plan

### 1. Theory

- Word → Vector (Vocabulary, tokens, embeddings)
- Sequence-to-sequence modeling
- Transformer basics

### 2. Dataset & Vocabulary

### 3. Tokenization & Special Tokens ( `<sos>`, `<eos>`, `<pad>` )

### 4. Simple Transformer Model

### 5. Training

### 6. Inference (Generating new responses)

### 7. Assignments for deeper practice

## 2. Theory

### 2.1. Word → Vector (Embeddings)

- Why do we need word vectors?
  - Neural networks operate on numbers, not text.
  - We convert words to dense vectors (embeddings) so that semantically similar words have similar vector representations.
- Vocabulary
  - We create a set (or list) of known words from our conversation data (e.g., "hello", "how", "are", "you", "i'm", etc.).
  - Each unique word is given a **token ID** (e.g., word "hello" → token ID 4 ).

## 2. Theory

- Adding Special Tokens
  - `<sos>` : start of sentence.
  - `<eos>` : end of sentence.
  - `<pad>` : for padding to a fixed length.
  - `<unk>` : unknown words not in our vocab.

## 2. Theory

### 2.2. Sequence-to-Sequence (Seq2Seq)

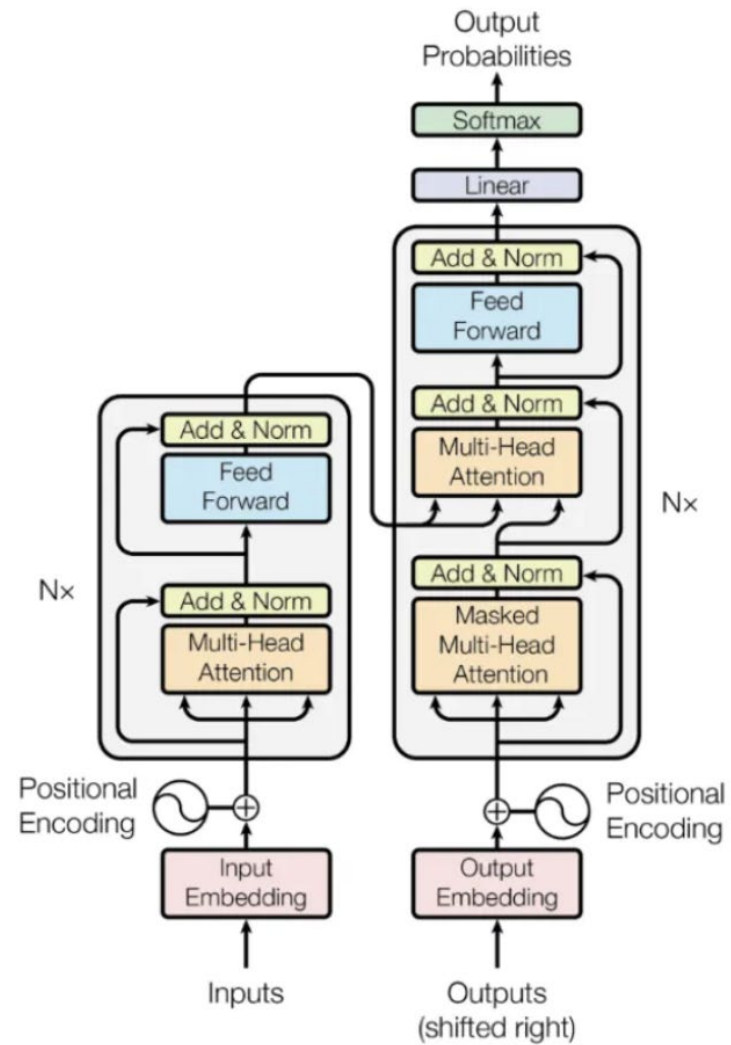
- **Concept:** We feed an **input sentence** to an **encoder** which processes it into a hidden representation. A **decoder** then processes that representation (and previous tokens it generated) to **predict the next token**.
- **Key:** The decoder continues generating tokens until it outputs `<eos>`.

## 2. Theory

### 2.3. Transformer Basics

- Uses **attention** to see different parts of the sentence simultaneously.
- Works well for translations, conversation generation, etc.
- **Encoder**: reads input sequence.
- **Decoder**: generates output sequence, one token at a time, while looking at the encoder's hidden representation.
- We'll implement a **simplified** version for demonstration.

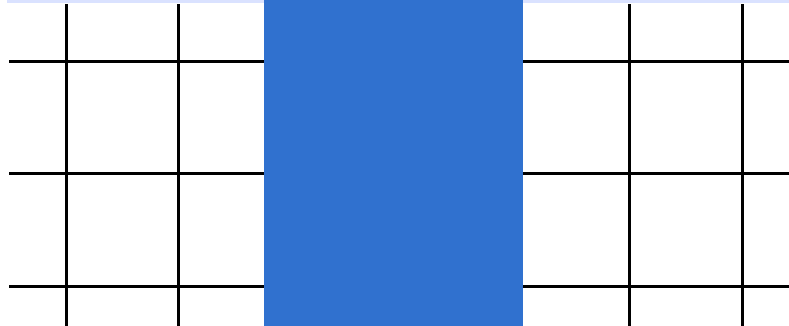




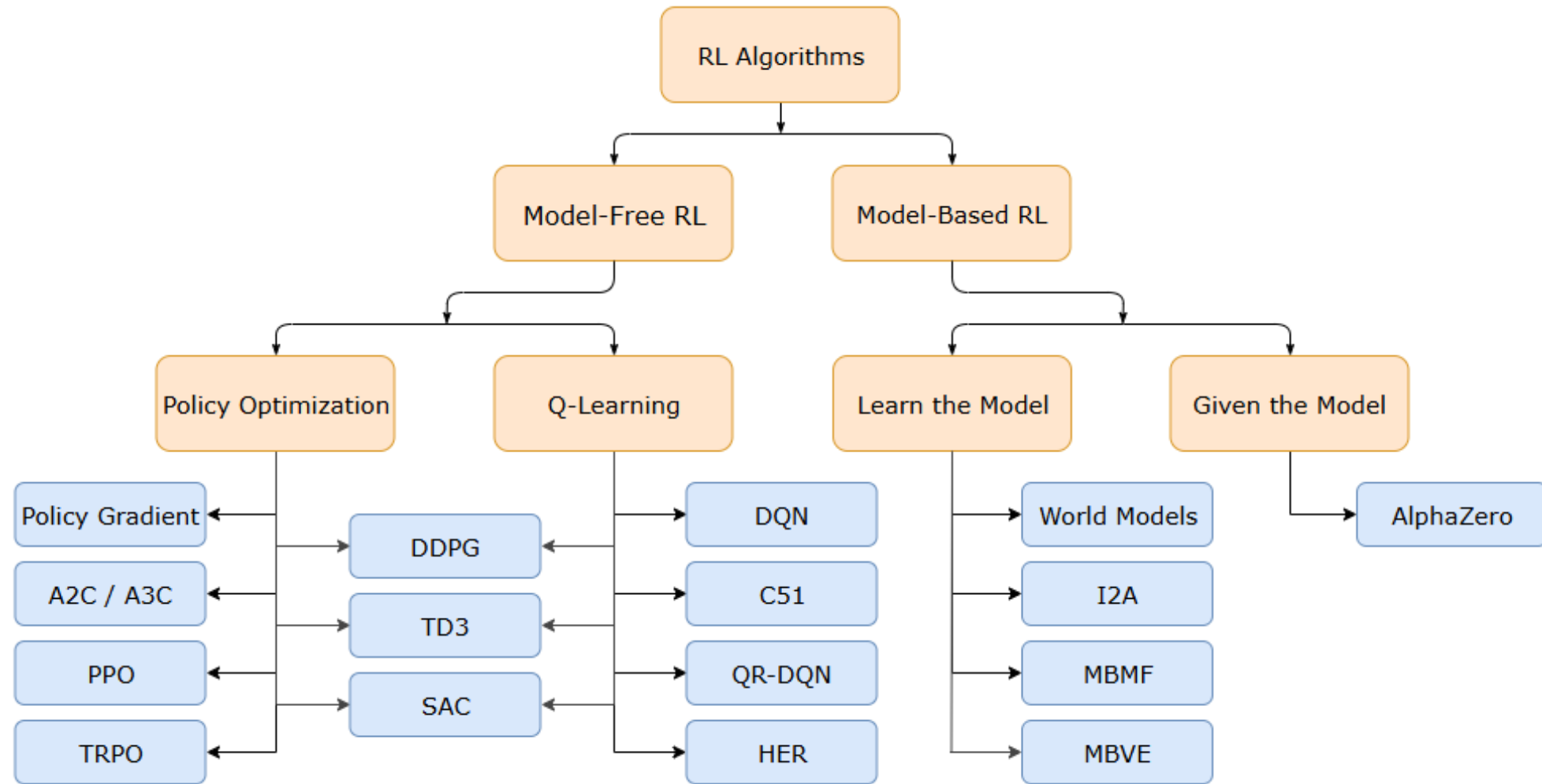
Link: <https://medium.com/@sayedebad.777/building-a-transformer-from-scratch-a-step-by-step-guide-a3df0aeb7c9a>

# INTRO REINFORCEMENT LEARNING (RL)

– Matee Vadrukchid –



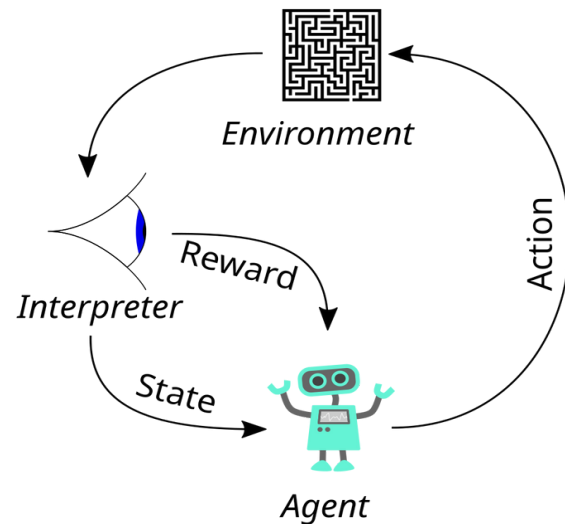
# Big Picture of RL



# Introduction and Outline

## What is Reinforcement Learning?

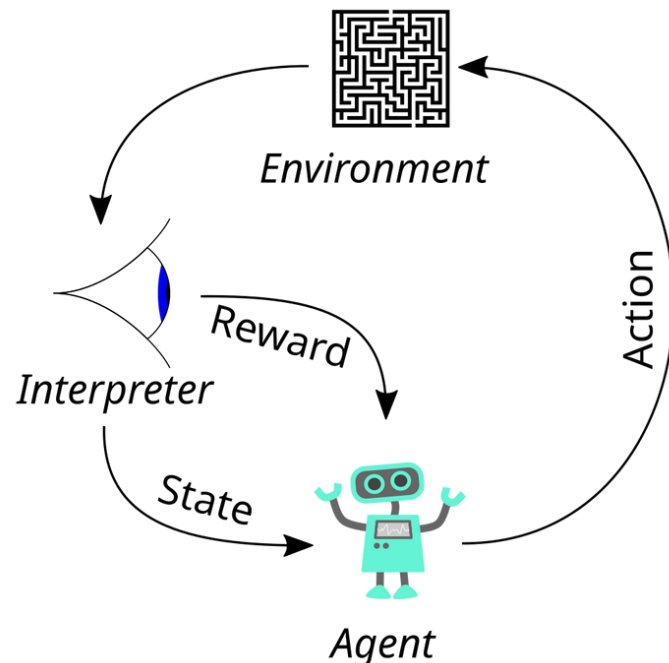
- In Reinforcement Learning, an agent interacts with an environment to learn how to perform a particular task well



# Introduction and Outline

## What is Reinforcement Learning?

- How is it different to the other learning paradigms?
  - There is no supervisor, only a reward.
  - The agent's actions affect the subsequent data it receives
  - Feedback is delayed, and may be received after several actions



# Introduction and Outline

## Cat Agent

State: Sitting



Action: walk



Action: keep sitting



Stay hungry..

Observable

Come here!

Reward



# Introduction and Outline

## Examples of Reinforcement Learning

Fly a helicopter



Make a robot walk



Manage an investment portfolio

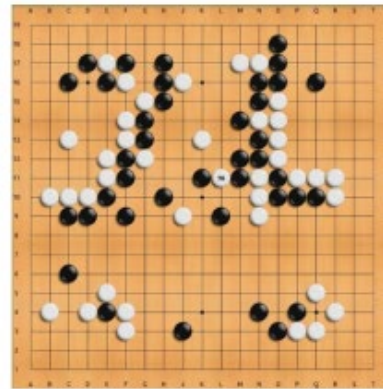


Play Atari games better than humans

# Introduction and Outline

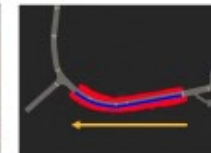
## Example of games

- Chess



- Go

- Carla





# Explanation & Theory

## Overview

- **GridWorld** is a simple, discrete environment where the world is represented as a grid. Each cell in the grid represents a state.
- **States and Actions:**
  - **States:** In our code, each cell is defined by its (row, col) coordinate. For example, in a 4×4 grid there are 16 states.
  - **Actions:** The agent can take one of four actions: Up, Down, Left, or Right.
- **Start and Goal:**
  - The **Start** state is fixed (top-left cell, i.e., `(0, 0)` ).
  - The **Goal** state is set to the bottom-right cell `(rows - 1, cols - 1)` .
- **Obstacles:**
  - Certain cells (for instance, `(1,1)` ) are designated as obstacles.
  - If the agent tries to move into an obstacle, it receives a penalty (in this version, the episode terminates with a reward of -1).

# Explanation & Theory

## Out-of-Bound Actions

- If the agent attempts to move off the grid (e.g., moving up from the top row), the environment immediately gives a penalty of -1 and terminates the episode.
- This teaches the agent that certain actions are “illegal” and should be avoided.

## Reward Structure

- **Goal:** Reaching the goal gives a reward of +1.
- **Invalid Actions:**
  - Moving out-of-bound or hitting an obstacle gives a reward of -1 and terminates the episode.
- **Neutral Moves:** Valid moves that do not result in hitting an obstacle or reaching the goal provide a reward of 0.

# The Q-Learning Algorithm

## Key Concepts

- **Q-Value:**
  - The Q-value  $Q(s, a)$  estimates the expected cumulative reward the agent will receive by taking action  $a$  in state  $s$  and then following an optimal policy.
- **Q-Table:**
  - A table where each row corresponds to a state and each column corresponds to an action.
  - The goal of Q-learning is to learn this table by iteratively updating it.

# The Q-Learning Algorithm

## The Update Rule

The core update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left( r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

- $\alpha$  (learning rate) controls how much new information overrides old information.
- $r$  is the immediate reward received after taking action  $a$  in state  $s$ .
- $\gamma$  (discount factor) determines how much future rewards are valued relative to immediate rewards.
- $\max_{a'} Q(s', a')$  is the estimated maximum future reward from the next state  $s'$ .

# The Q-Learning Algorithm

## Epsilon-Greedy Exploration

- **Exploration vs. Exploitation:**
  - With probability  $\epsilon$ , the agent chooses a random action (exploration) to gather more information about the environment.
  - With probability  $1 - \epsilon$ , the agent chooses the action with the highest Q-value (exploitation) to maximize its reward.
- **Epsilon Decay:**
  - Over time,  $\epsilon$  decays, meaning the agent gradually shifts from exploration to exploitation as it becomes more confident in its learned Q-values.